1 Network mode

This section documents the multi-station network estimator (`ProcessingMode::PPP_RTK_Network`), which *produces* the satellite-side corrections that the single-receiver user modes consume. The other modes in this reference either difference away the satellite clock and hardware biases (the differencing section), take them as known inputs from precise orbit, clock and bias products, or import them as known SSR quantities (the SSR-corrections section). The network estimator inverts that relationship. It estimates the satellite clock and the satellite *phase* bias as explicit states, and the satellite *code* bias as well when `network.estimate_sat_code_bias` is set (armed by both reference configurations; the loader default is off), jointly across a network of stations with known coordinates, and emits them in IGS-SSR convention for a downstream user. The loop is closed by design: the single-receiver PPP-RTK engine documented elsewhere is the product test, run on a *test station* held out of the network solve.

Scope and status. The numerical core consists of the arrowhead normal-equation kernel (`network/solve/NormalEquations`) and the semantic estimator that drives it (`network/solve/Estimator`). It is validated on a synthetic noise-free network, where it recovers the truth up to the declared S-transform (`tests/network/test_network_estimator.cpp`), and on real and simulated networks through the two reference configurations (`config/golden/golden_field_refound.json`: 58 stations of real data, day 129 of 2024; `config/golden/golden_sim_refound.json`: 60 stations of a simulated data set). Both are GPS L1/L2 and Galileo E1/E5a, dual frequency, with a free per-line-of-sight slant ionosphere (no global ionosphere map, no spherical-harmonic constraint), the satellite geometry-free code bias pinned at the broadcast group delay, the prioritized phase datum with a held basis, the float ambiguity vehicle, and the armed quality control: post-fit network RAIM with its attribution pass and within-epoch tier (the network-RAIM section), the gross line-of-sight screen, the measurement admission screen and the clock runaway check. The network owns its stations' parameter sets (Section 1.2) and builds every row through the one observation equation that also serves the single-receiver filter and the first-epoch seeds (the one-equation section). The pipeline around the core is complete: the `PPP_RTK_Network` enum with the forced uncombined configuration of the PPP-RTK family, the explicit satellite phase-bias state (Section 1.3), the mode dispatch in `Application::run`, the setup calls and epoch source of `Application::runNetwork`, and one epoch as `Application::networkProcessEpoch_`, which calls a fixed, ordered sequence of single-concern seams (Section 1.1). Every step derived in this chapter runs inside one of those seams.

This chapter treats the single-epoch solve and its datum. The between-epoch carry has three layers; each is developed in its own document.

- The *shared border* (satellite clocks, phase biases, code biases) is always carried as a filtered solve. In the reference configurations the carried posterior is one dense information block $(\boldsymbol{P}_{\text{prev}} + \boldsymbol{Q})^{-1}$ (`network.shared_carry_full_cov`, `networkInjectD2CarryPrior` in `network/state/BorderPrior`). With that setting off, each key is carried as a diagonal prior of variance $\text{var} + q\,\Delta t$. A satellite clock takes the increment of the temporal clock anchor instead when `network.clock_temporal_anchor` is on (both reference configurations), and a phase or code bias variance is floored when a floor is set. A clock-pivot handover drops the carry of the constellation whose datum changed (`network.scoped_handover_response`), unless `network.handover_stransform` re-expresses it in the incoming datum (Section 1.5); both settings are on in the reference configurations.

- The *station-local columns* are carried as whitened coupling rows of each station's design, not as independent priors (the between-epoch carry document). In the reference configurations the carried families are the float ambiguities (the float ambiguity vehicle, `network.ambiguity_vehicle` set to `float_kf`; the loader default is `level`) and the receiver code biases (`network.carry_rx_code_bias_in_block`). The vehicle, its exact coupling time update (`network.carry_coupling_exact_update`) and the temporal clock anchor are developed in the float-ambiguity-vehicle section.
- The *invariant DD carry* (`network.carry_invariant`, default off, off in the reference configurations) carries the double-differenced cycle basis instead of the raw ambiguities (the cycle-space section).

The event responses and the holds on the emitted product are in the event-response and emitted-product section. The single-difference extra-widelane constraint `network.sd_ewl`, a PASS B row set that is not a layer of the carry, is default off.

1.1 Architecture: the decomposed epoch loop

The rest of this chapter is mathematics. This subsection maps each piece of it to the code that executes it, which is spread over several translation units.

Decomposition. The network driver began as a single function of more than 14,000 lines. At that size it could not be reasoned about as a whole, exercised below the granularity of a full run, or reviewed: a search for the datum build could not bound where it was *not*. It was decomposed into single-concern *seams*, first by verbatim relocation gated on byte-identical output against the pre-cut binary (per-increment records: `docs/record/network_decomposition_plan.md`), then into one source folder per concern under `src/core/network/` and `include/network/`: `driver`, `state`, `assembly`, `carry`, `solve`, `qc`, `slip`, `ar`, `emit`, `diag` and `runend`. The driver (`network/driver/ApplicationNetwork.cpp`) holds the setup calls and the epoch source. One epoch is the spine (`network/driver/ApplicationNetworkEpoch.cpp`). The placement rules for new network code (the anti-regrowth decision tree in `CLAUDE.md`) exist to keep both short.

Four stages. A network run has four stages: three translation units and the seam sequence between them.

- **Setup** (`network/driver/ApplicationNetworkSetup`). `networkValidateInputs_` rejects a station with no observation `rinex_path` before anything is allocated. `networkSetup_` then builds the station roster and its synchroniser, the product sink and its declared S-basis contract, and, when the closed loop is armed, the test-station engine (Section 1.11). Its output is one `NetworkSetupResult`, filled in place and neither copyable nor movable: three run-long references pin its member addresses, so the member declaration order in that header (`network/driver/SetupResult.hpp`) is normative. Further setup builders, one per concern and called in a fixed order from `runNetwork`, configure the engines the run owns.
- **Assembly** (`network/driver/ApplicationNetworkAssembly`), two passes per epoch. PASS A is *layout*. It runs each present station's geometry models and pre-allocates the shared satellite states. On the legacy single-pivot datum (`network.phase_datum_prioritized` off, the loader default) it takes the datum (pivot) station *first*, so that the phase-bias columns admitted to the shared block are those backed by a pivot-receiver ambiguity pin, and the local half of the datum (Section 1.4) therefore constrains the column layout as well as the solve. Under the prioritized datum that the reference configurations arm, the pivot is not taken first and the

shared set is the union elected by the phase-anchor epoch (Section 1.6). PASS B is *assembly*. Each station's undifferenced design block is computed in parallel over the stations (`network.parallel_stations`; `networkAssemblyProduce`), with no receiver-side differencing, and then *split* (`networkSplitStation`): the satellite clock and satellite-bias columns scatter into the shared border $\boldsymbol{A}_{s,r}$, and the remainder, less the columns held at their seed and the structurally unobservable ones, becomes the private block $\boldsymbol{A}_{r,r}$ of Equation (1). The three receiver-position columns are dropped, because network station coordinates are held fixed; the split throws if one reaches $\boldsymbol{A}_{r,r}$. The blocks are then folded into the estimator serially in station order (`networkAssemblyConsume`). Equation (9) is formed from these two pieces.

- **The epoch seams**: the ordered sequence tabulated below, which is the body of the loop.
- **Teardown** (`network/runend/Teardown`). The run-end readout is one report per subsystem, each a row of `kNetworkRunEndReports` (`network/runend/RunEnd`). `networkTeardown` walks that table and returns the process exit code; the driver ends with a `return` of that call. The service counters (`NetworkServiceCounters`, `network/state/RunContext`) and the summary writers (`NetworkRunTraces`, `network/diag/RunTraces`) are held outside the teardown, which reads them.

The seam contract. A seam is a free function (or one of five `Application` members) over the run, `NetworkRun&`, and the epoch, `const NetworkEpoch&`, with any outputs as further parameters. The run (`network/driver/Run.hpp`) holds the references to the context, the configuration, the logger, the trace bundle and the setup, and owns the estimation engines, the emit engines, the timing record and the closed-loop tally. The epoch (`network/driver/Epoch.hpp`) holds the values the stages hand each other, grouped by the stage that writes them; it is built empty with the epoch and destroyed at its end, so nothing in it reaches the next epoch. A seam *owns no state*: every carried object is a reference into the run. It opens with one binding per name it uses, reads through the epoch, and writes only through its output parameters. A block that used to `continue` the epoch loop returns an `EpochAction` of `Proceed`, `SkipEpoch` or `StopRun` (`network/driver/RunTypes`) instead; the per-skip bookkeeping stays inside the seam. A block whose locals are read further down the loop returns a result struct (`NetworkSolveGlueResult`, `network/solve/SolveGlue`) that the spine stores in a group of the epoch (`ep.solve`); later seams read the values there. The placement of a new epoch-level value, event or observer is fixed by the same decision tree: a value the next epoch reads is run state, never an epoch member.

The run context. Carried state, configuration snapshots and counters live in `NetworkRunContext` (`network/state/RunContext`). It holds a `const NetworkKnobs` snapshot of the configuration, the cross-epoch `NetworkCarryState`, the entity tables (`ctx.entities`, `network/state/Entities`) and `NetworkServiceCounters`. The entity tables key the cross-epoch state by typed identity (station, constellation, line of sight, satellite and signal, satellite, arc; `common/GnssIdentity`), and each field has a declared persistence row (`kStationFields` and its siblings). A cross-epoch value outside the entities has a row in `net::kRunFields` (`network/state/RunFields`); an undeclared member is rejected by the verification script `scripts/nom_n4_verify.py`. Persistence follows the *three-place rule*: a carry field that is a snapshot member appears in the warm-resume fingerprint, in the restore and in the snapshot write, in `network/state/SnapshotFingerprint`, `network/state/WarmResume` and `network/state/StateSnapshotWrite` respectively. A field present in only two of the three yields a warm resume that disagrees with the run that wrote it, with no error raised.

The ordered epoch seams. The table below lists the per-epoch call sequence in spine order (`networkProcessEpoch_`, `network/driver/ApplicationNetworkEpoch`). The spine's section timers bracket groups of the calls. Rows marked *default off* are inert unless armed. The measure-only instruments (the tallies, the frame diagnostics, the first-epoch tables) are not spine calls. Each is a row of the table `kNetworkEpochObservers` (`network/driver/EpochObservers`, 22 rows in six slots), and the spine walks a slot with `networkObserveEpoch`; the row after the table summarises them.

seam call (spine order)	concern (translation unit)
`networkFastForward`, `networkInjectEpoch`, `networkCanonicalPairAdmission`, `networkSecondBandAdmission`	start-offset skip; simulated-fault injection (default off); record admission upstream of every model and detector: the canonical-pair rule and the second-band rule (both default off; Section 1.10) (`network/driver/EpochAdmission`, `network/carry/Inject`, `network/assembly/CanonicalPairAdmission`, `SecondBandAdmission`)
`networkBuildSignalRoster`, `networkAdmitEpoch`	signal roster and arc ledger, advanced on every synced epoch; the epoch's clears and broadcast feeds, the max-duration stop, the decimation skip (`network/assembly/SignalRoster`, `network/driver/EpochAdmission`)
`networkSlipFunnelOpen`, `networkPassAHead_`, `networkPassASlipScan_`, `networkSlipFunnelUnionRaw`	PASS A: the slip funnel opens with the previous epoch's deferred flags; per-station geometry and shared-state pre-allocation, on the legacy datum the datum station first; the slip scans that fill the funnel (`network/slip/SlipFunnel`, `network/driver/ApplicationNetworkAssembly`, `network/slip/PassASlipScan`, `SlipQc`)
`networkAssemblyPrefilter_`	raw slip observables; cross-station observability prune (`network/driver/ApplicationNetworkAssembly`)
`networkSignalIntegrity`, `networkMeasurementAdmission`	per-line-of-sight band-state classifier (default off); product-referenced corrupt-channel screen, the one admission test a differential screen cannot see (armed by `network.adm_screen`) (`network/assembly/SignalIntegrity`, `MeasurementAdmission`)
`networkPhaseAnchorEpoch`, `networkReceiverPhaseBiasAllocation`	election of the *local* half of the phase datum: the candidate tally and coverage gate, then the armed realisation (`network/solve/PhaseAnchorEpoch`, `PhaseDatum`, `AnchorHygiene`, `ChainDatum`); the receiver phase-bias allocation is inert unless the chain datum is armed
`networkAssemblyRegisterColumns_`	clear the layout, register this epoch's shared columns, open the arrowhead system (`network/driver/ApplicationNetworkAssembly`)
`networkSdEwlAr`, `networkOraclePinEpoch`, `networkAmbFloatDropEpoch`	single-difference extra-widelane decision (default off); simulation-only ambiguity pin; the float vehicle's bounded drop (`network/ar/SdEwlAr`, `network/slip/OraclePin`, `network/ar/AmbFloat`)
`networkClockLevelRestoreRaise`, `networkHandoverStransformBeforeRows`, `networkCarryDecide`, `networkCouplingExactPrepare`	the first two default off; the fold decision for every carried border entry, made once per epoch and read by both the coupling rows and the border fold; the inputs of the exact coupling time update (`network/state/CarryDisposition`, `network/state/BorderPrior`)
`networkPassB_`	PASS B produce and consume: the split design blocks handed to `addStation` (`network/driver/ApplicationNetworkAssembly`, `network/assembly/StationBuild`, `network/carry/Coupling`)
`networkGapResumeCensus`, `networkCarryDriverBeginEpoch`, `networkBuildCarryDdPriors`	arc-resumption tally (measure-only); the invariant DD carry's predict and prior blocks (default off) (`network/slip/GapResumeCensus`, `network/carry/DdPriorBuild`)
`networkSolveGlue`	the solve as a fixed list of 14 stages (below) (`network/solve/SolveGlue`, calling `Sbasis`, `CodeBiasDatum`, `ClockDatum`, `RankGate`, `SbasisCompletion`, `network/qc/RaimWithinEpoch`)

seam call (spine order)	concern (translation unit)
networkInstallCarry, networkProducerAr	DD posterior read-back into the invariant carry (default off); network-side integer decision, read-only and log-only unless its fixed-cycle fold is set to emit (default off) (network/carry/InstallCarry, network/ar/ProducerAr)
networkGrossLosScreen (decision half), networkFixedCycles, networkSolPrep	gross line-of-sight decision, so a declaration reaches the epoch it declares (network.gross_los_emit_exclude); fixed-cycle fold (default off); map to the solution epoch and per-satellite discontinuity signalling (network/qc/GrossLos, network/ar/FixedCycles, network/emit/SolPrep)
networkPbHoldCloneUpdate, networkEmitStore	smoothed-hold clock clones (default off); emit re-base, phase-bias hold, store push, product write, test-station step (network/emit/PbHoldClone, EmitStore, PbHold)
networkIonoDiag, networkDdFrac, networkShDevVtec, networkShHealth, networkLeveling	diagnostic sweeps, and the harvest of the station-local carries after the solve (networkHarvestLocalCarries, network/carry/LocalCarryHarvest) (network/diag/IonoDiag, DdFrac, ShDevVtec, ShHealth, Leveling)
networkPostFitRaim, networkGrossLosScreen (response half)	post-fit network RAIM with its attribution pass; the gross line-of-sight response (network/qc/Raim, GrossLos)
networkClockQc	post-solve satellite-clock quality control and the retro-flag feed (network/qc/ClockQc)
networkSignalRosterAudit, networkBlockBaardaPredict, networkCongruencePredict	roster disagreement pass; the predict halves of the block-Baarda and station-congruence tallies (network/assembly/SignalRoster, network/qc/BlockBaarda, Congruence)
networkStashStatus, networkPbHoldCloneHarvest, networkClockLevelMonitor, networkBufferedTraceDrains	carry re-stash and the epoch-status record; the clones' re-harvest and the level monitor (both default off); the buffered trace rows (network/diag/StashStatus, network/emit/PbHoldClone, network/solve/ClockLevel)
networkWriteStateSnapshot	warm-resume state snapshot (network/state/StateSnapshotWrite)
networkObserveEpoch, six slots	the 22 measure-only instruments, by slot: admitted (satellite-identity tally), before columns (level-authority tally), after clock QC (phase-visible monitor), roster tail (ledger emit, second-phase and all-channel censuses, second-constellation census, the *parameters as armed* and *contributions as armed* tables), frame diagnostics (post-fit variance factor and residual traces, reduced-normal spectrum, station-block audit, prefit census, redundancy census, block-Baarda and congruence drains, minimal-detectable-bias flush, datum-graph census, reduced-RHS attribution and its flight recorder), stash tail (carry-state trace drain, entity reach sampler) (network/driver/EpochObservers)

The solve. networkSolveGlue (network/solve/SolveGlue.hpp) calls 14 stages in a fixed order; the estimator accumulates its folds in that order, so the order is part of the numerical result. In order: (1) networkCensusSigma0, the prefit variance tally; (2) networkCodeBiasDatum, the satellite code-bias datum rows (Section 1.7); (3) networkClockDatum, the clock datum and pivot election (Section 1.5); (4) networkPreGatePriors; (5) networkEstimabilityDump; (6) networkKaulaMargin; (7) networkSbasisCompletion; (8) networkRankGate (Section 1.8); (9) networkClockModelWarm; (10) networkPostGatePriors (the ephemeris re-level, the handover S-transform, the border carry prior, the clock-oscillator prior and the cold clock anchor); (11) networkPrimarySolve; (12) networkClockAnchorGuardEpoch; (13) networkWithinEpochAdapt, the within-epoch RAIM tier; (14) networkSigma0Ceiling. Four stages can end the epoch, five ways; each way is a row of kSolveSkips. Everything the rank gate must see is folded before stage 8.

Table 2: Parameter families of a network station. The kind names are those of `NetworkParamKind` (`common/ParamRef`).

family	kind	content
receiver clock and inter-system biases	`rx_clock`	$c\,\mathrm{d}t_r$ and one $c\,\delta t_r^{(g)}$ per active constellation g other than the time reference (GPS)
wet troposphere	`zwd`	ZWD_r (no gradient columns)
slant ionosphere	`slant_iono`	one $I^i_{r,1}$ per tracked satellite, scaled to signal s by μ_s (+ on code, − on phase)
ambiguities	`ambiguity`	one float ambiguity per satellite and phase signal, $\lambda_s N^i_{r,s}$ in metres
receiver code biases	`rx_code_bias`	differential bias of the constellation's clock-reference pair, inter-frequency bias of each further band, inter-signal correction of a second code
receiver phase biases	`rx_phase_bias`	$\delta_{r,s}$ per (system, signal); only under the chain datum (`network.phase_datum_eq34`, default off)
phase colour	`phase_colour`	per-satellite phase nuisance, Gauss–Markov (`network.phase_colour_state`, default off)
position	`rx_position`	three columns, held at the roster coordinate

Reading key. The derivations that follow map to places in that sequence. The rank deficiency and its minimum-constraint datum (Section 1.4 to Section 1.8) are built and gated inside `networkSolveGlue`. The arrowhead Schur reduction (Section 1.9) runs in the kernel that the same seam invokes, on normals accumulated station by station during PASS B. The emitted product and the test station (Section 1.11) are handled in `networkEmitStore`. The rest of the table is assembly, datum election, ambiguity resolution and solution preparation, with quality control, diagnostics and carry bookkeeping, around those places.

1.2 Joint multi-station observation model

The network estimator uses the same *undifferenced, uncombined* observations as single-receiver PPP (the observation-equations section), but stacks the equations of every station $r \in \mathcal{R}$ into one system. The unknowns partition into a block *shared* by all stations and a block *private* to each:

- **Shared (satellite-side).** The estimable satellite clock $c\,\mathrm{d}t^i$, the per-signal satellite code bias B^i_s, and the per-signal satellite *phase* bias $\bar{b}^i_s$. These columns are identical for every station, since a satellite's clock and hardware delays do not depend on the observer. Orbits are held fixed to a precise product; orbit determination is outside the scope of the network estimator. A fourth family, the coefficients of a global spherical-harmonic VTEC expansion (`network.estimate_iono_sh`), is kept in the code and is off in both reference configurations: the ionosphere is the free per-line-of-sight slant delay below (`docs/refound/AB1_sh_consumer_product_2026-09-20.md`).
- **Private (station-side).** For each station r the parameter set of Table 2. Station coordinates are known and held fixed, so position is not a free parameter.

The parameter set. A station's model is a list of columns, held in one `StationParams` (`network/state/StationParams`). Each column has a typed identity (`ParamRef`, `common/ParamRef`), a linearization point and the disposition the split gave it (the network-station-parameters section of the state-vector document describes the layout from the state-vector side). The set also names the shared border parameters the station's rows reference and the shared parameters a gate withheld from it. Table 2 lists the families.

Dispositions and the *parameters as armed* table. Every column of every station, and every shared satellite parameter a gate withheld from a station, is an instance with a declared *disposition* on each epoch (`ParamDisposition`, `network/assembly/ParamInventory`): *estimated in the shared border*, *estimated in the local block*, *held at its seed* (in the station's model at a fixed value, outside the solve), *absorbed* (a shared satellite phase or code bias a gate withheld from the station, which another column of the station takes up), *pinned* (the station coordinates), *dropped with a stated reason* and *unobserved* (in the layout, but its design column is zero this epoch). The run prints the counts per parameter class and disposition as the *parameters as armed* table once after the first emitted epoch and again at the run end, with a warning for a class governed by a rule whose premise does not hold, and the run manifest carries the same record. A gate that withholds a shared parameter has a row in a reason table (`AbsorbedReason`): the GLONASS code-only scope, the coverage allowlist of the phase-bias columns, the legacy pivot allowlist, the code-bias admission tally, the anchored-union coverage gate and the anchor-hygiene block. On both reference configurations the border count plus the absorbed count of a satellite phase-bias class equals the ambiguity count of the same signal, because the screens that remove an observation remove its ambiguity with it. A second table, *model as armed*, is built from the parameter registry (`NetworkParamRegistry`, `network/assembly/ParamRegistry`), which declares once per parameter kind whether it is estimated and carried, its process-noise rule and rate, its variance floors, its gap horizon, the route it takes into the next epoch's solve (dense border, diagonal border, local block, conditional row, diagonal station prior) and the responses it has to slips, gaps, handovers, decoupling and re-admission.

Write the shared satellite-parameter vector as $\boldsymbol{x}_s$ and station r's private nuisance vector as $\boldsymbol{x}_r$. The undifferenced observation rows of station r linearise to

$$\boldsymbol{y}_r \;=\; \boldsymbol{A}_{s,r}\,\boldsymbol{x}_s \;+\; \boldsymbol{A}_{r,r}\,\boldsymbol{x}_r \;+\; \boldsymbol{\varepsilon}_r, \qquad r \in \mathcal{R}, \tag{1}$$

where $\boldsymbol{y}_r$ is the prefit residual (observed minus computed), $\boldsymbol{A}_{s,r}$ is the Jacobian with respect to the *shared* satellite columns, and $\boldsymbol{A}_{r,r}$ is the Jacobian with respect to station r's *own* nuisances. There is no $\boldsymbol{A}_{r,r'}$ term for $r' \neq r$: a station's observations touch only the shared block and its own private block. The reduction of Section 1.9 rests on that sparsity.

One equation. A row's predicted observable is the term list of `observationEquation` (`measurement/ObservationEquation.hpp`; the one-equation section). The network reads it through `NetView`, which answers for each parameter with the slot of the station's set and its linearization point (`networkStationRows`, `network/assembly/StationRows`). A shared satellite parameter answers with its border column and the value zero: it is absolute in the network solve and is never written back to a station. On the network's rows the satellite clock enters with the partial -1 (`kSatClockPartial`) and each satellite bias, code or phase, with $+1$ (`kSatBiasSign`); the same two constants multiply the known terms of a user of the corrections. A network station always has one ambiguity per band, and the network takes no SSR bias term, because it estimates the biases themselves. The estimated bias parameters therefore carry the sign of the term as it appears in the observation, $-B_s^i$ and $-b_s^i$ of the pseudorange equation and the carrier-phase equation.

No receiver-side differencing. Unlike the PPP-RTK user mode, the network estimator applies *no* between-satellite or between-receiver differencing. Its rows are the undifferenced equation of `networkStationRows`, and the differencing strategy of the mode is `DifferencingMode::None` (`differencingModeFor` in `measurement/MeasurementBuilder.cpp`), overriding any `between_satellite_sd` toggle in a network configuration. Differencing would cancel the satellite-side terms

the network *estimates*. The network carries them as explicit columns instead and resolves the resulting rank deficiency with a minimum-constraint datum (Section 1.4).

1.3 The explicit satellite phase-bias state

The main contrast with the user side is the treatment of the satellite phase bias b_s^i (written $\bar{b}_s^i$ once promoted to a state). On the user side this term is never an explicit unknown: between-satellite single differencing (the differencing section) cancels the *receiver* phase bias, the satellite phase bias arrives as a *known* SSR product, and what remains is an integer single-difference ambiguity. The user handles biases *implicitly*.

The network estimator *generates* that known product, so it carries the satellite phase bias as an estimated state. The shared border gains a per-(satellite, signal) phase-bias column,

$$\bar{b}_s^i \in \boldsymbol{x}_s, \qquad \text{registered by } \texttt{NetworkEstimator::registerPhaseBias}(i, s), \quad \text{layout key } \texttt{"}i\texttt{|PB|}s\texttt{"}, \tag{2}$$

disjoint from the satellite *code*-bias columns (layout key `"`i`|CB|`s`"`) and the clock column (`"`i`|CLK"`). A station's parameter set names the parameter by the typed identity of kind `sat_phase_bias`; the spelling `"`i`|`s`|PB"` of `filter/StateKeySchema` is the single-receiver filter's key for the same family (the state-vector section). Its process noise is a small random walk (`network.process_noise.sat_phase_bias`, legacy `filter.process_noise.sat_phase_bias_q`; default $1 \times 10^{-12}\,\mathrm{m^2\,s^{-1}}$, two orders of magnitude below the `sat_code_bias` default of $1 \times 10^{-10}\,\mathrm{m^2\,s^{-1}}$). The rate is the carry rate of the border, applied as $\mathrm{var} + q\,\Delta t$ in the carried prior. Satellite phase hardware delays are nearly constant but not strictly so: a $q = 0$ random walk would over-converge the diagonal of $\boldsymbol{P}$ and leave the column insensitive to real slow drift (the registry kind `sat_phase_bias`, `network/assembly/ParamRegistry`).

Remark 1. The estimated $\bar{b}_s^i$ is not the raw satellite phase hardware delay, which is not separately estimable. It is the delay *lumped with the network pivot-receiver ambiguity/clock datum* (Odijk's δ^s, [1]): the estimable combination that, when removed by a user receiver, restores the integer property of that user's single-difference ambiguities. Because of this lumping the phase-bias product must be *consistent* (the SSR-corrections section): the datum of the network and the integer recovery of the user must use the same S-system.

1.4 Rank deficiency and the minimum-constraint S-basis

The undifferenced network system Equation (1) is rank deficient. The linear dependencies that the user side removes by differencing appear here as exact *gauge freedoms* of the joint normal matrix. No observation distinguishes a common shift of all satellite clocks of a constellation from the opposite shift of the receivers' clocks (the receiver clock for the time-reference constellation, the inter-system bias for each other one); none fixes the split of an absolute phase bias between the satellite column and the per-station ambiguity datum; none fixes the common level of the code biases between the satellite and receiver sides, or their split between the clock, the ionosphere and the geometry-free combination. These are the S-system ambiguities of [2]: the data fix only *estimable combinations*, and a *datum* must be chosen to pin the remaining null space. The gauges are, in the order the datum treats them: one clock gauge per constellation; one phase-bias against ambiguity gauge per (satellite, signal); and the code-bias gauges of Section 1.7.

Exact gauge, soft elimination. The datum is a *minimum-constraint* datum applied as exact hard pins to the reduced shared system, never a structural elimination of columns. Each shared

gauge that is a pin is realised as a single *Dirichlet pin* of one shared column c to a datum value (almost always 0): the column's known contribution moves to the right-hand side, its row and column in the reduced normal matrix are zeroed, and a diagonal entry is set so that $x_c =$ value holds exactly (`NetworkEstimator::solve`, `applyDatumPins` in `network/solve/Estimator.cpp`). The pin is applied to a *copy* of the reduced normals (the kernel's `solveSharedConstrained` seam). The estimator's own reduced system stays unmodified, so per-station back-substitution (Section 1.9) stays valid. The pinned diagonal is set to the reduced normals' own scale (the largest diagonal magnitude, at least one) rather than to unity. An *under*-specified datum then still appears as a near-zero *relative* pivot in the positive-definiteness guard, independent of the network's overall weighting magnitude. The gauges that are rows (code-bias gauges, the sum-to-zero and broadcast-anchored clock datums, the S-basis completion) are folded as tight constraint rows before the rank gate; their weights are listed in the *contributions as armed* table (`network/assembly/Contributions`).

In S-transform language, let $\boldsymbol{x}$ collect the rank-deficient undifferenced parameters and $\boldsymbol{S}$ be the S-basis selection. The emitted satellite corrections are the estimable combination $\boldsymbol{S}\,\boldsymbol{x}$. Choosing the datum is choosing $\boldsymbol{S}$, and the pins and rows above are the minimal set of rows of $\boldsymbol{S}$ that remove the deficiency. The datum is declared in the product (`SbasisDeclaration`, Section 1.11) and has two halves that live in two places:

- **Shared-block gauges** touch the shared satellite columns: the clock datum, one per constellation, and the code-bias gauges. The clock pins go through the explicit `NetworkEstimator::Datum` (a list of column pins) and are applied in `solve`.
- **Local datum elements** live on one station's private block: the phase-bias gauge is realised by tight prior rows on the ambiguities of the elected anchor station, which the emitted satellite phase bias absorbs (Odijk Eq. 4/9). The *caller* folds them into that station's $\boldsymbol{A}_{r,r}$, $\boldsymbol{W}$, and $\boldsymbol{y}$ at information 10^{10} (row kind `ambiguity_pin`), not through the shared `Datum`. Station priors also make each local block $\boldsymbol{N}_{rr}$ full rank for the Schur step (Section 1.9).

Selectable datum realisations. Each gauge admits alternatives, selected by configuration and recorded in the `SbasisDeclaration`. All produce the same estimable $\boldsymbol{S}\,\boldsymbol{x}$ and differ in how they distribute the gauge. Table 3 lists what the reference configurations arm and what the loader offers instead.
The removed ensemble realisation of the phase gauge (a sum-to-zero over the receivers) remains in the S-basis declaration so that a stored product naming it can be read, and its Schur-projected fold remains a kernel capability (the three-doors section); no configuration the loader accepts arms it.

1.5 The clock datum and the pivot election

The pivot pin. Adding a constant to every satellite clock of a constellation and absorbing it in the receivers' clock and inter-system bias leaves every row unchanged. In the default pin mode (`SbasisDeclaration::IsbDatum::PerConstellationSatClockPin`) the gauge is fixed by one hard pin per constellation: the clock correction of one satellite, the *pivot*, is held at 0, that is at the a priori product clock, and every other clock of the constellation is estimated against it. No system is anchored to a receiver clock; each station's clock and inter-system bias stay free local states. After the local ambiguity pins are eliminated, the reduced shared normals then have exactly one null direction per constellation, which is the declared S-basis dimension that the rank gate checks (Section 1.8). The two alternatives are constraint-realised and need no pivot: the sum-to-zero mode replaces the pin by the tight row $\sum_i \Delta\delta t^i = 0$ over the constellation's registered clocks, and the broadcast-anchored mode anchors each clock to its own a priori value by a soft pseudo-observation

Table 3: Datum realisations. The reference configurations are `config/golden/golden_field_refound.json` and `config/golden/golden_sim_refound.json`.

gauge	reference configurations	other realisations (default status)
clock, per constellation	hard pin of one satellite clock, the pivot; sticky incumbent (`network.sticky_pivot`); Section 1.5	sum-to-zero over the constellation (`network.clock_datum_sum_to_zero`); per-satellite soft anchor to the a priori clock (`network.clock_datum_broadcast_anchored`); declared pivot (`network.clock_datum_satellite`); coverage election (`network.pivot_election`)
phase bias against ambiguity, per (satellite, signal)	prioritized anchor election, held basis (`network.phase_datum_prioritized`, `network.phase_datum_held_basis`); Section 1.6	one pivot receiver (the loader default); single-receiver chain (`network.phase_datum_eq34`, off)
satellite code biases	ionosphere-free pin, geometry-free pin and sum-to-zero rows, time-linked, geometry-free pin at the broadcast group delay (`network.sat_gf_datum`); Galileo family difference; Section 1.7	geometry-free pin at zero or none; Kaula prior (`network.cb_kaula_prior`, off)
receiver code biases	loose zero-mean prior on each station's differential bias (`network.rx_code_bias_datum`)	none

of declared 1σ. In both the declared dimension is 0. The reference configurations keep the pin. `network.clock_datum_station` names the reference receiver of the declaration; the pin is on a satellite clock, never on that receiver's clock.

Choosing the pivot. The pivot is only a gauge, but a satellite that few stations track, that the broadcast flags unhealthy, or that enters the Earth's shadow soon, holds the whole constellation's clock level on a weak and short-lived tie. The choice per constellation, in priority order (`NetworkSbasis::electClockPivot`):

1. the declared pin `network.clock_datum_satellite`, kept even when unhealthy;
2. the sticky incumbent (`network.sticky_pivot`, armed in the reference configurations), while it stays registered and, under `network.sat_health`, healthy: a new pivot is chosen only when the incumbent is absent, and the old one returning does not flip the datum back;
3. the pick of an earlier election that no emitted product has adopted yet;
4. a new pick: the first registered satellite in lexical order or, with `network.pivot_election` set to `coverage`, the satellite the coverage ranking below names.

Election by predicted coverage. The election (`network/solve/PivotElection`) applies at a constellation's first epoch and at a forced handover only (the incumbent left the registered set, or the broadcast flags it unhealthy). It reads no observation: it uses the orbit product, the roster coordinates and the Sun. For a candidate satellite i with an orbit and the samples $t_k = t_0 + k\,\Delta$, $k = 0, \ldots, K$, with $K = \lfloor H/\Delta \rfloor$, horizon H (`network.pivot_election_horizon_s`, default 7200 s) and step Δ (`network.pivot_election_step_s`, default 300 s), the score is

$$n_i \;=\; \Big|\Big\{(k,r) \;:\; r \in \mathcal{R},\; 0 \le k \le K,\; \theta_r^i(t_k) \ge \theta_{\min}\Big\}\Big|, \tag{3}$$

with $\theta_{\min}$ the run's elevation mask (`processing.elevation_mask_deg`), and the umbra time of i is the first sample inside the cylindrical Earth shadow (radius scaled by `GnssConfig::getEclipseShadowScale`). Candidates that stay out of the umbra for the whole horizon come first, and the largest n_i wins. When every candidate enters the umbra, the one that enters it last wins, then the larger n_i. A tie goes to the lexically first satellite, and a candidate without an orbit is never elected. The ranking is a pure function of integer counts, so the preview before PASS B and the solve glue, which both elect, always agree. The setting is default off, never armed, and inert under the two constraint-realised clock datums (`docs/refound/PIV_pivot_election_coverage_2026-10-17.md`).

Handover. When the pin moves from satellite p to q inside a constellation, the estimable clock solution of the whole constellation is re-referenced. The pin swap is a rank-one S-transform along the constellation's structural clock null direction v_c:

$$c_0 = \frac{x_q^{\text{pre}}}{v_c[q]}, \qquad x_i \leftarrow x_i - v_c[i]\, c_0, \qquad \text{var}_i \leftarrow \text{var}_i + v_c[i]^2\, \text{var}_q, \tag{4}$$

where x_q^{pre} is the carried level of the incoming pivot in the outgoing frame (`network/solve/HandoverStransform`). Only the constellation's clock columns have a nonzero component of v_c, and every between-satellite difference is unchanged to machine precision. With `network.handover_stransform` the carried clocks are re-expressed this way and folded; without it the carry of the changed constellation is skipped for one epoch and re-levels from the data. The code-bias and phase-bias carries are released to fold unchanged, because their gauges are realised independently of the clock pivot. The variance update drops the cross term, which is conservative for the positively correlated clock common mode; the full $\boldsymbol{SPS}^{\mathrm{T}}$ form is a separate default-off setting (`network.handover_stransform_dense`). The time-linked clock anchor and the clock step test that bind a satellite's carried clock across epochs are in the float-ambiguity-vehicle section.

1.6 The phase datum

The satellite phase bias $\bar{b}_s^i$ is collinear with the column of ambiguities $\{N_{r,s}^i\}_r$ (null vector $\delta\bar{b}_s^i = -\delta N_{r,s}^i$ for any per-satellite shift). The datum fixes this gauge once per (satellite, signal) key by pinning the ambiguity of one station. The emitted phase bias therefore carries that station's ambiguity lump (Section 1.3).

Legacy single pivot. By default the datum receiver is one pivot receiver, whose ambiguities are all pinned (at information 10^{10}), and the shared phase-bias set is restricted to the keys that pivot tracks. A satellite low or unseen at the pivot then has no clean gauge.

Prioritized anchors. With `network.phase_datum_prioritized` (armed in the reference configurations) the shared set is the *union* of the keys that at least `network.phase_datum_min_stations` stations track (the coverage gate; a key below the bar is pruned at every station and, with `network.phase_datum_gate_drop_erase`, its carried ambiguities are erased at the drop edge). For each key one tracking station is the *anchor*: its ambiguity is pinned and the other stations' are free. This is the same one-pin-per-gauge constraint realised at a different station per key (`networkBuildPhaseDatumCandidates` and `networkElectPhaseAnchors`, `network/solve/PhaseDatum`). The election keeps the sticky incumbent while it is present, still tracks the key and is not deposed by a screen flag on its own column (`network.phase_datum_sticky`, default on).

Otherwise, under the held basis, the candidates whose arc is old enough (`network.phase_anchor_min_age_ep`) and has been free of screen flags for `network.phase_anchor_flag_quiet_ep` epochs are ranked by descending elevation, and the highest one whose carried level passes the handover ledger test is elected. When none passes, the highest eligible one is elected and pinned at 0, a cold handover, and when none is eligible the coverage-ranked sweeps over the trackers elect (coverage is counted above `network.phase_datum_pref_elev_deg`, default 15°). A handover pins the incoming anchor at its own carried level (`network.phase_datum_handover_continuous`), so the family's frame does not step. The ledger test refuses a successor with no ledger entry, a non-finite value, or a carried variance that is not finite, not positive or above `network.phase_datum_handover_max_var_m2`; a cold handover steps the emitted phase bias, and it is declared to the user (`network.phase_datum_held_mark_cold`). The *held basis* (`network.phase_datum_held_basis`) freezes the pin value of a retained incumbent for its whole tenure, so the datum is constant between declared handovers (the float-ambiguity-vehicle section). The election runs once per epoch after the admission screens, whose withholds prune the states an election could otherwise pin (`network/solve/PhaseAnchorEpoch`).

Chain datum. With `network.phase_datum_eq34` (off) the gauge is fixed by one datum-receiver phase-bias pin plus a spanning tree of ambiguity pins over the bipartite receiver–satellite graph per (constellation, signal) connected component, with per-(receiver, system, signal) phase-bias split states $\delta_{r,s}$ that carry each receiver's fractional bias (`network/solve/ChainDatum`). It is the substrate of the invariant DD carry (the cycle-space section).

1.7 The satellite code-bias datum

With `network.estimate_sat_code_bias` the network estimates a code bias B^i_s per satellite and code signal, and the external `.BIA` product is not used. The code biases are collinear with the clock, the slant ionosphere and the receiver code biases. The datum rows are built by `NetworkSbasis::buildCodeBiasDatumRows`, folded by `networkCodeBiasDatum` (stage 2 of the solve) as tight rows of information 10^3, before the rank gate. With no ionosphere constraint (`network.w6_iono_constraint none`, armed in the reference configurations) these rows are the whole code-bias datum.

Per-satellite pins. For a satellite with code signals s_1, s_2 (the two lowest distinct-frequency signals of its registered set, chosen deterministically), two rows are built. The ionosphere-free pin ties the clock-common gauge,

$$a_1\, B^i_{s_1} + a_2\, B^i_{s_2} = 0, \qquad a_1 = \frac{f_1^2}{f_1^2 - f_2^2}, \quad a_2 = -\frac{f_2^2}{f_1^2 - f_2^2}, \tag{5}$$

so the ionosphere-free lump lives in the estimable satellite clock. The geometry-free pin fixes the split that the slant ionosphere would otherwise absorb,

$$\frac{B^i_{s_2} - B^i_{s_1}}{\mu_{s_2} - \mu_{s_1}} \;=\; d^i_{\mathrm{GF}}, \qquad \mu_s = \frac{f_1^2}{f_s^2}, \tag{6}$$

with d^i_{GF} a datum quantity: any value is legal and the estimable parameters absorb the difference. `network.sat_gf_datum` selects it: `none`, `zero`, or `broadcast_tgd` (pinned in both reference configurations), where $d^i_{\mathrm{GF}} = c\,T_{\mathrm{GD}}$ for GPS $(L1, L2)$ and $c\,\mathrm{BGD}(E1, E5a)$ for Galileo $(E1, E5a)$. The identity is exact: in the model's sign convention the broadcast biases are $cT_{\mathrm{GD}}\,\mu_s$, whose ionosphere-free combination is identically zero and whose geometry-free combination is cT_{GD}, so the two pins do not conflict. The target is used only when the pinned pair is the pair the group

delay describes; any other pair falls back to zero and is counted. Extra signals of a satellite are pinned through their geometry-free combination against the band-1 reference at the same target, and a channel whose every column the datum pins has its sum-to-zero row suppressed.

Level of each channel. The common level of a channel (g, s) between the satellite and the receiver sides is fixed by a sum-to-zero row over the satellites that carry it. On the receiver side the same freedom is fixed by a loose zero-mean prior $\delta_r \sim N(0, \sigma^2)$ on each station's differential bias (`network.rx_code_bias_datum`, armed; $\sigma =$ `network.rx_code_bias_datum_sigma_m`, default 10 m). The prior adds $1/\sigma^2$ along the common mode, independent of the station count, and leaves the between-station differences to the data.

Time linking. With `network.code_datum_time_linked` (armed) the rows pin the *change* of the datum instead of its level. The right-hand side of the ionosphere-free row becomes the satellite's previous solved combination $a_1 \hat{B}_{s_1,\mathrm{prev}} + a_2 \hat{B}_{s_2,\mathrm{prev}}$, but only when the pair is unchanged and both signals have previous values; otherwise the row re-seeds at 0 and the satellite is marked. The geometry-free rows carry their previous combination (`network.sat_gf_datum_time_linked`), with the absolute target only at a first sighting. The sum-to-zero row spans the channel's *veterans* (satellites with a previous solved value) with right-hand side $\sum_{\mathrm{vet}} \hat{B}^i_{s,\mathrm{prev}}$, so a rise or set does not move the target. A channel with no veteran re-seeds at $\sum = 0$, which is the one path on which the emitted level re-references; it is counted and declared.

Family difference. Galileo band-1 and band-5 codes come in two families by tracking attribute: the combined I+Q correlator output (attribute `X`, the *secondary* family) and the discrete components (the pilot, the *primary* family; `channelFamilyOf`). Their biases are distinct hardware paths with their own bias. A time-linked sum-to-zero row per family would leave the two families' levels independent, so that the combined and pilot clocks differ by a datum accident. With `network.code_datum_family_difference` (armed with `network.code_datum_time_linked`) the sum-to-zero row of a secondary channel X is replaced by a row over the satellites $\mathcal{C}$ that carry both X and its primary partner P of the same band,

$$\sum_{i \in \mathcal{C}} \left(B^i_X - B^i_P \right) \;=\; \sum_{i \in \mathcal{C}_{\mathrm{vet}}} \left(\hat{B}^i_X - \hat{B}^i_P \right)_{\mathrm{prev}} , \tag{7}$$

time-linked like the sum-to-zero row. The partner is the same-band primary channel with the most satellites in common. The sum-to-zero row is kept where no partner or common satellite exists, where every spanned secondary column is pinned by another row, and for one epoch when the family row has no veteran but the sum-to-zero row has. At the first sighting of a secondary or partner column, with `network.code_family_birth_prior_sigma_m` (armed at 0.10 m), one soft row $B^i_X - B^i_P = \mu$ is folded, with μ the family's veteran mean difference and 0 without a veteran. The option `network.code_datum_pair_continuity`, which holds the ionosphere-free pin pair through a gap, is off: it preserved the two families' birth realisations and worsened pilot-family users (`docs/record/refound/GFC_galileo_family_clock_datum_2026-10-10.md`, `docs/record/refound/GAD_galileo_adoption_2026-10-13.md`).

1.8 The rank gate and the S-basis completion

A null-space rank gate eigen-decomposes the pre-constraint reduced normals and asserts that the count of small eigenvalues equals the *declared* S-basis dimension. It is implemented as

`NetworkSbasis::assertNullSpaceRank` and is invoked on `reducedSharedNormals()` *before* the constrained solve, against the declared dimension of the clock datum's plan. An eigenvalue is small when it is below $\mathrm{tol} \cdot \max(\lambda_{\max}, 1)$, with tol = `network.rank_gate_tol` (loader default 10^{-9}, 10^{-11} in the reference configurations). The call sits in stage 8 of the solve (`networkRankGate`, `network/solve/RankGate`). The gate complements the positive-definiteness guard. The guard catches the exact-deficiency (under-specified datum) case on the pinned pivot; the rank gate checks the reduced deficiency against the declared gauge dimension in both directions and records whether a mismatch was an under- or an over-specified datum, which need opposite fixes.

Two mechanisms keep a sound epoch from being withheld by a weak column. *S-basis completion* (`network.sbasis_complete`, armed; stage 7, `networkSbasisCompletion`) constrains each separated extra near-null direction of the reduced system along its own eigenvector at its carried level, with a tight row of information 10^3, instead of withholding. It is capped per epoch (`network.sbasis_complete_max`, default 6), refuses when the weight is less than two decades above the cutoff, and skips a direction not separated by a decade from the declared-gauge cluster; both edges are events. Under the prioritized phase datum the gate then runs a *drop-and-heal* pass that soft-anchors the near-null phase-bias columns toward 0 at $\sigma = 1\,\mathrm{m}$ (`healNearNullPhaseBias`, `network/solve/PhaseDatum`) and re-gates once. On the legacy pivot datum there is nothing to heal and the epoch is withheld immediately.

1.9 Per-station Schur reduction (arrowhead normals)

Stacking Equation (1) over all stations and forming weighted normal equations $\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\boldsymbol{A}\,\boldsymbol{x} = \boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\boldsymbol{y}$ yields a *bordered block-arrow* ("arrowhead") system. The station–station off-diagonal blocks are exactly zero, because stations couple only through $\boldsymbol{x}_s$:

$$\begin{bmatrix} \boldsymbol{N}_{ss} & \boldsymbol{N}_{s1} & \boldsymbol{N}_{s2} & \cdots & \boldsymbol{N}_{sR} \\ \boldsymbol{N}_{1s} & \boldsymbol{N}_{11} & \boldsymbol{0} & \cdots & \boldsymbol{0} \\ \boldsymbol{N}_{2s} & \boldsymbol{0} & \boldsymbol{N}_{22} & \cdots & \boldsymbol{0} \\ \vdots & & & \ddots & \\ \boldsymbol{N}_{Rs} & \boldsymbol{0} & \boldsymbol{0} & \cdots & \boldsymbol{N}_{RR} \end{bmatrix} \begin{bmatrix} \boldsymbol{x}_s \\ \boldsymbol{x}_1 \\ \boldsymbol{x}_2 \\ \vdots \\ \boldsymbol{x}_R \end{bmatrix} = \begin{bmatrix} \boldsymbol{u}_s \\ \boldsymbol{u}_1 \\ \boldsymbol{u}_2 \\ \vdots \\ \boldsymbol{u}_R \end{bmatrix}, \tag{8}$$

with the per-station blocks built from that station's pre-formed design rows,

$$\boldsymbol{N}_{ss}^{r} = \boldsymbol{A}_{s,r}^{\mathrm{T}}\boldsymbol{W}_r\boldsymbol{A}_{s,r}, \quad \boldsymbol{N}_{sr}^{r} = \boldsymbol{A}_{s,r}^{\mathrm{T}}\boldsymbol{W}_r\boldsymbol{A}_{r,r}, \quad \boldsymbol{N}_{rr}^{r} = \boldsymbol{A}_{r,r}^{\mathrm{T}}\boldsymbol{W}_r\boldsymbol{A}_{r,r}, \tag{9}$$

and $\boldsymbol{u}_s^{r} = \boldsymbol{A}_{s,r}^{\mathrm{T}}\boldsymbol{W}_r\boldsymbol{y}_r$, $\boldsymbol{u}_r^{r} = \boldsymbol{A}_{r,r}^{\mathrm{T}}\boldsymbol{W}_r\boldsymbol{y}_r$. A full (non-diagonal) $\boldsymbol{W}_r$ is supported, so correlated undifferenced measurement covariance (e.g. shared pre-corrections) propagates exactly (`addStationObservations`, `network/solve/NormalEquations.hpp`). The station's diagonal prior rows can be passed to `addStation` so that the positive-definiteness test judges the block the solve uses (`network.station_prior_before_pd`, armed): a station whose observation-only block is rank deficient in the direction its own datum row gauges is then kept.

Lossless marginalisation. The arrowhead system is solved by Schur-eliminating each station's private block onto the shared border. Each $\boldsymbol{N}_{rr}^{r}$ is factorised once ($\mathrm{LDL}^{\mathrm{T}}$) when the station is added, and `reduce` and `reconstructStation` reuse the factor. The elimination accumulates

$$\tilde{\boldsymbol{N}}_{ss} = \sum_{r\in\mathcal{R}} \left(\boldsymbol{N}_{ss}^{r} - \boldsymbol{N}_{sr}^{r}\,(\boldsymbol{N}_{rr}^{r})^{-1}\,\boldsymbol{N}_{sr}^{r\mathrm{T}} \right) \;+\; \boldsymbol{N}_{ss}^{\mathrm{prior}}, \tag{10}$$

$$\tilde{\boldsymbol{u}}_s = \sum_{r\in\mathcal{R}} \left(\boldsymbol{u}_s^{r} - \boldsymbol{N}_{sr}^{r}\,(\boldsymbol{N}_{rr}^{r})^{-1}\,\boldsymbol{u}_r^{r} \right) \;+\; \boldsymbol{u}_s^{\mathrm{prior}}, \tag{11}$$

the reduced shared system $\tilde{\boldsymbol{N}}_{ss}\,\boldsymbol{x}_s = \tilde{\boldsymbol{u}}_s$ (`NetworkNormalEquations::reduce`). The reduction is a *lossless* marginalisation. Solving Equations (10) and (11) for $\boldsymbol{x}_s$ and back-substituting per station,

$$\boldsymbol{x}_r = (\boldsymbol{N}_{rr}^{r})^{-1}\big(\boldsymbol{u}_r^{r} - \boldsymbol{N}_{sr}^{r\mathrm{T}}\,\boldsymbol{x}_s\big), \tag{12}$$

reproduces the dense stacked solve to machine precision. Fill-in is confined to the satellite border and each station's elimination is independent, so the cost is linear in the station count and the per-station eliminations run in parallel (`reconstructStation`, `network/solve/NormalEquations.cpp`). The shared-block priors $\boldsymbol{N}_{ss}^{\mathrm{prior}}$ in Equation (10) (soft physical priors, the datum rows and the carried border prior) are added before the reduction through `addSharedInfo` and pass through the Schur complement unchanged.

Exact sparse station normals. A station's design rows touch a handful of the n_s shared columns (its satellites' clocks and biases) and of its own locals. The dense products of Equation (9) spend almost all of their work on zeros. With `network.sparse_station_normals` (default on, set in both reference configurations; `false` gives the dense products) the blocks are formed from the nonzero entries only (`NetworkNormalEquations::formStationNormalsSparse`). The result is not an approximation. Each element is summed in the order Eigen's blocked product sums it: the terms in ascending row order from +0, except that an element of a 4-column block in the 2-row tile of its row panel keeps two accumulators (even and odd row index) that are added before the remaining rows. A skipped term is an exact zero, so the blocks equal the dense products bit for bit. The function reports that equality is not guaranteed, and the caller then forms the products densely, when the build contracts multiply and add, when Eigen runs more than one thread (the driver pins it to one for the epoch loop), when a result is a vector or the shapes are so small that Eigen evaluates the product coefficient by coefficient, when the row count exceeds the panel depth, and when an input, weight or weighted entry is not finite. The shared block $\boldsymbol{N}_{ss}^{r}$ is accumulated on the square of the columns any row touches and added to the border entry by entry, which replaces an $n_s \times n_s$ product and symmetry check per station and epoch. The equality is gated against Eigen's own product in `tests/network/test_neq_sparse_station_normals.cpp` and was checked on whole runs (`docs/record/refound/NETCOST_gps_galileo_cost_attribution_2026-10-18.md`).

The datum frame, and the covariance of the locals. The covariance of the shared estimate is the inverse of the datum-constrained reduced normals, $\boldsymbol{\Sigma}_{ss} = \tilde{\boldsymbol{N}}_{ss}^{-1}$ of the constrained system, recovered from the same $\mathrm{LDL}^{\mathrm{T}}$ factor as the solve (`solveSharedConstrained`). A datum-pinned column and a folded ensemble gauge column carry only an artificial variance, so the consumers of the covariance treat them as exact. The *datum frame* $\boldsymbol{\Sigma}_{ss}^{\mathrm{df}}$ is $\boldsymbol{\Sigma}_{ss}$ with the rows and columns of those columns zeroed. A station's locals then have the covariance

$$\mathrm{Cov}(\boldsymbol{x}_r) \;=\; (\boldsymbol{N}_{rr}^{r})^{-1} \;+\; \boldsymbol{M}_r\,\boldsymbol{\Sigma}_{ss}^{\mathrm{df}}\,\boldsymbol{M}_r^{\mathrm{T}}, \qquad \boldsymbol{M}_r = (\boldsymbol{N}_{rr}^{r})^{-1}\boldsymbol{N}_{sr}^{r\mathrm{T}}, \tag{13}$$

which every reader of the datum-frame covariance uses: the conditional variance and regression of each carried column (the carry harvest), the row redundancy of the RAIM and Baarda tests, the redundancy and block tallies, and the covariance of a carried double difference (the posterior and innovation section). The frame is a function of three inputs only (the last solve's $\boldsymbol{\Sigma}_{ss}$, the pinned columns of the datum and the folded gauges), so the solve builds it once, right after $\boldsymbol{\Sigma}_{ss}$, and stores it with the list of pinned columns. A reader whose datum pins the same columns in the same order reads the stored matrix; any other datum builds a local frame by the same function. The stored frame is invalidated at the start of the next solve, by a reset and by a change of the gauges, and

only the non-const solve writes it, so the parallel station loops read immutable data. This replaced one copy of the n_s^2 matrix per call, and it changes no output bit.

The *border-known variance* is the case $\boldsymbol{\Sigma}_{ss}^{\mathrm{df}} = \mathbf{0}$ of Equation (13): $\mathrm{diag}\big((\boldsymbol{N}_{rr}^{r})^{-1}\big)$, the variance of a local column with the whole shared border treated as known. The anchor-handover test uses it, because the uncertainty of the shared phase-bias column moves the pin target and the realised frame together and so is common mode in a handover step. It is computed by `stationConditionalVarianceDiag`, one local solve $n_c = (\boldsymbol{N}_{rr}^{r})^{-1} e_c$ per column, without forming or multiplying the all-zero $n_s \times n_s$ matrix. The result is bit-identical to the product form: with $m_c = \boldsymbol{N}_{sr} n_c$ finite, $m_c^{\mathrm{T}} \mathbf{0}\, m_c = \pm 0$ leaves a nonzero variance unchanged, and the two cases where the sum could differ (a zero variance, a non-finite m_c) take the product itself (`docs/record/refound/CVF_conditional_variance_frame_2026-10-29.md`).

Remark 2. This arrowhead topology with exact reduction mirrors the NGS M-PAGES `NgsLsq::NormalEquations::reduce` / `reconstructLocals` batch network solver. In the specialisation, the M-PAGES "global" block is the shared satellite border and the per-epoch swept block becomes the R independent station blocks. A two-stage covariance-form alternative was rejected because the normal-equation Schur form is exact and cheaper here.

1.10 Constellation scope

The network estimator supports GPS, Galileo and BeiDou on the same footing, selected by `processing.active_systems`. GPS is the time reference: it carries no inter-system bias, and every other active constellation adds one per station. The reference configurations are GPS and Galileo.

Galileo. The reference configurations estimate E1 and E5a for Galileo beside L1 and L2 for GPS (`processing.frequency_mode dual`), with both code families of each Galileo band on the signal list (`processing.signal_priorities.Galileo`: `C1C`, `C1X`, `L1C`, `L1X`, `C5Q`, `C5X`, `L5Q`, `L5X`). The package that makes this posture work includes the code-bias family difference (Section 1.7), the receiver cross-system antenna fallback (`model_corrections.enable_receiver_cross_system_atx_fallback`, which substitutes the nearest GPS band when the receiver antenna entry lacks a Galileo frequency) and, on the user side, `estimation.clock_disc_isb_reinflate_std` of 10 m: a network clock-datum re-reference steps one constellation's satellite clocks by a common constant, which the receiver clock absorbs for the reference constellation but which lands on the near-frozen inter-system bias of any other, so the user re-inflates that bias on a consumed clock-datum discontinuity (`docs/record/refound/GAD_galileo_adoption_2026-10-13.md`).

Free slant ionosphere. The ionosphere of the network posture is the free per-line-of-sight slant delay $I_{r,1}^{i}$ of each station, re-estimated every epoch, with no external global map and no spherical-harmonic constraint (`estimation.ionosphere_weighting.enable` off, `network.w6_iono_constraint none`). The datum of the code biases is therefore the pins above and nothing else (Section 1.7).

BeiDou and the second-band rule. BeiDou runs under the same rules (code-bias datum, phase datum, clock pivot) and is not armed in the reference configurations. On B1I and B3I it runs healthy on the 58-station real data set: every epoch is emitted and none is withheld. A station that tracks a non-reference constellation on one band only leaves three directions of its local block collinear: the constellation's inter-system bias, the slant ionosphere of its lines of sight and their ambiguities.

One band gives a code row and a phase row per satellite against two free unknowns, so a common shift of the bias is absorbed by the unknowns of every satellite. The positive-definiteness guard of `addStation` then rejects the whole station at every epoch, and its other constellations leave the solve with it. The second-band rule (`network.second_band_admission`, default off) withholds the records of a constellation at a station unless the station holds it on two modelled bands. A band is modelled for a satellite when the epoch carries an admitted code signal and an admitted phase signal on it. Under `network.second_band_scope station` (default) the constellation is admitted at the station while at least `network.second_band_min_sats` of its satellites are on two modelled bands there, and under `satellite` each satellite is tested alone. The unit that fails is withheld whole, upstream of the roster, the models and the detectors, so it is absent as if the RINEX record were missing and its return is read as a re-entry. Each edge is an event (`network/assembly/SecondBandAdmission`). The time-reference constellation is outside the rule. With BeiDou on B1I and B3I the rule reproduces the subtraction of the single-band stations; the setting is unvalidated and not armed (`docs/record/refound/BDS_beidou_third_constellation_2026-10-18.md`, `docs/record/refound/BDS305_beidou_305_stations_6h_2026-10-29.md`).

Third band: the receiver band-5 code delay. When the satellite code biases are estimated, the receiver inter-frequency bias columns of the bands outside a constellation's clock-reference pair would absorb the code information that separates that band's satellite clock from its phase bias. The declared rule (`RxIfbRule`, `network/assembly/ParamInventory`) therefore leaves them out of the station's local block, in the station's model at the value their first-epoch seed gave them (disposition *held at seed*). The rule rests on a premise, that an ionosphere weighting leaves each station's ionosphere level free to fold the band's receiver delay; without that weighting nothing takes the delay up, and the *parameters as armed* table prints a warning. With `network.estimate_rx_extra_band_code_delay` (default off, unvalidated) the columns are estimated in the local block and carried by the receiver code-bias carry. The setting acts only where a constellation tracks a band outside a two-band pair under the run's frequency mode, so it is inert on the dual-frequency reference configurations (`docs/refound/L5B_band5_receiver_delay_2026-10-10.md`).

GLONASS. GLONASS phase biases are excluded. The reason is structural: GLONASS is FDMA, so each satellite transmits on a slightly different frequency, and the receiver inter-channel (hardware) bias does *not* cancel across satellites as it does for the CDMA constellations. The satellite/receiver phase-bias bookkeeping that underpins the explicit-bias S-system (Sections 1.3 and 1.4) assumes a per-signal bias common to a constellation's satellites. The per-frequency GLONASS inter-channel bias breaks that assumption and would corrupt the phase datum. GLONASS is therefore excluded from the network *phase*-bias product by design (`docs/record/attic/ssystem_conformance_2026-06-15.md`); the user side excludes it from ambiguity resolution for the same reason. A *code-only* GLONASS mode is available behind `network.estimate_glonass` (`enableGlonassNetwork`, default off, kept off). It admits GLONASS to the solve with a satellite clock and a per-slot receiver inter-frequency bias but *no* FDMA phase bias, so the phase-datum bookkeeping above is unaffected. The mode is mutually exclusive with the satellite code-bias slice: GLONASS admission skips the GPS/Galileo receiver DCB that code-bias separation needs, so `network.estimate_sat_code_bias` together with `network.estimate_glonass` is a hard load error. This mode is distinct from the float-only GLONASS support that the user engine carries for positioning. With the code-only mode off, the network estimator generates no GLONASS bias products.

Broadcast health. Broadcast satellite health enters the network path through `network.sat_health` (`off`, `pivot` or `exclude`, default `off`; unvalidated, not armed). Under `exclude`, setup removes a satellite's observations from the synchronised epochs at which the broadcast flags it; under `pivot`, only the clock-pivot election reads the health series (Section 1.5).

1.11 The emitted product and the test station

The emitted product. The network emits one `NetworkSolutionEpoch` per content epoch: the per-satellite clock C_0 and the per-signal code and phase biases, keyed by RINEX satellite id and serialised in IGS-SSR convention by `NetworkProductEmitter` (`NetworkEstimator::toSolutionEpoch`). The product also declares its datum (the S record, `SbasisDeclaration`): the reference receiver, the clock pivot of each constellation, the convention of the phase-bias gauge, the code-bias datum, the sign of the biases, and the wind-up model revision (`windup_id`, `kWindUpYawModelId`). A user checks the declaration before it claims integer recovery (`NetworkProductReader::checkContract`), so a product built under a different wind-up convention is refused. Network and user share one wind-up convention, the accumulated wind-up of the `nominal-wu93` frame, and a mismatch is absorbed by the ambiguities and is invisible to the post-fit variance factor.

A user applies the product as known corrections: the satellite clock correction with the factor -1 and each signal bias with the factor $+1$, the constants the network's own rows use (the one-equation section), restoring the integer ambiguity property as described in the SSR-corrections section.

The test station. The validation follows from this symmetry. The companion `ProcessingMode::PPP_RTK_User` mode runs the ordinary uncombined PPP-RTK pipeline but takes the network's own bias and clock product. A station held *out* of the network solve, with a known coordinate, and positioned as a user against the produced corrections is a closed-loop product test of the whole chain. With integer recovery on, its fixed ambiguities show whether the datum of the network and the integer recovery of the user are mutually consistent; with it off (the reference configurations set `network.closed_loop.ar_mode` to 0) its position error is the grade. The test station runs through the standard PPP path and is not a separate estimator; only the network estimator takes the joint multi-station pipeline of this section. The reference configurations hold out WTZZ00DEU (real data) and SZZ000SIM (simulated data).

Two forms of the test. The *in-process* form (`network.closed_loop.enable`) builds the user engine from a derived configuration during setup (`GnssConfig::deriveClosedLoopConsumerConfig`) and gives it an *owned* SSR correction store instead of the process-wide one, so the network's products cannot leak into a store that another component reads. The emit seam `networkEmitStore` steps the user after that epoch's products have been pushed, which closes the loop *within* the epoch. The user runs a between-satellite single-difference model by default, or an undifferenced one under `closed_loop.consumer_undifferenced`, which the reference configurations arm. The derived configuration is a full copy of the network's and leaves `estimation.accumulate_phase_windup` unchanged, so the in-process user runs the network's wind-up mode; the reference configurations arm accumulation on both beds. The *offline replay* form (`network.closed_loop.replay_products_path` → `Application::runConsumerReplay`, `app/ApplicationConsumerReplay`) short-circuits `runNetwork` before input validation and setup. It fills the same owned store from an already-written product file through the same reader path, so the recorded stream's discontinuity bookkeeping is reconstructed exactly, and a position-domain case costs minutes instead of a full run. The replay derives its user through the same call from the replay configuration, and the product's

S-basis contract records only the wind-up model revision, so a replay matches the network's mode only when its configuration does. The setting `network.closed_loop.consume_network_iono` would supply the user a slant ionosphere from a spherical-harmonic model; the reference configurations build none, so it supplies nothing.

Holds on the product. The reference configurations also arm two emit-side holds and the user's answer to them, treated in the event-response and emitted-product section. The phase-bias generation hold (`network.emit_pb_hold`) freezes each emitted phase bias at the first value of its generation and declares every new generation. The phase-clock hold (`network.emit_pb_hold_phase_clock`), a mode of the same hold, keeps the satellite's phase-visible clock over the generation instead. Both run inside `networkEmitStore` (`network/emit/PbHold`) and leave the solve unchanged. Under `network.closed_loop.pb_hold_presence_flip` the derived user (`processing.ssr_pb_hold_presence_flip`) answers a held phase bias that appears on, or leaves, a phase row it already uses as it answers an unheld one.

References

[1] D. Odijk. "Fast precise GPS positioning in the presence of ionospheric delays". In: *Publications on Geodesy* 52 (2003).

[2] P. J. G. Teunissen. "Generalized inverses, adjustment, the datum problem and S-transformations". In: *Optimization and Design of Geodetic Networks* (1985), pp. 11–55.